

1. Validating the Ingestion through API

Ans:- Validated the Health check, that server is running

The screenshot shows the Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "RAG MongoDB - Data Ingestion" which contains a sequence of requests: "1. Health Check", "2. Upload Excel - Test Cases", "2. Upload Excel - User Stories", "3. List Available Files", "4. Create Embeddings - Batch (Test Cases - FAST)", and "4. Create Embeddings - Batch (User Stories - FAST)". The main panel displays the "1. Health Check" request, a GET method to the endpoint "[(baseUri)]/api/health". The "Authorization" tab is active, showing "Inherit auth from parent". The response is a 200 OK status with a 30 ms response time and 312 B of data. The JSON body is:

```
{  "status": "OK",  "message": "Server is running"}
```

Converted test cases into Json

The screenshot shows the Postman interface with the same workspace. The left sidebar highlights the "2. Upload Excel - Test Cases" request. The main panel displays a POST request to the endpoint "[(baseUri)]/api/upload-excel". The "Body" tab is active, showing "form-data" with the following fields:

Key	Value	Description
file	testcases.xlsx	Excel file containing test cases
dataType	testcases	Type of data: 'testcases' or 'userstories'
sheetName	Testcases	Name of the Excel sheet to convert (optional)
Key	Value	Description

The response is a 200 OK status with a 1.70 s response time and 627 B of data. The JSON body is:

```
{  "success": true,  "message": "Test cases file converted successfully",  "outputFile": "converted-1782378818568.json",  "output": "⚠ Sheet '\\Testcases\\' not found. Using first sheet: '\\testcases\\'\\n✅ Converted 6354 rows from '\\testcases\\' into C:/Users/Nandhini/Shreekanth/Downloads/rag-mongo-demo-v9/rag-mongo-demo-v9/src/data/converted-1782378818568.json\\n"
```

Converted User Stories into Json

The screenshot shows the Postman interface for a REST client. The left sidebar displays a collection named 'RAG MongoDB - Data Ingestion' with several items, including 'POST 2. Upload Excel - User Stories' which is currently selected. The main panel shows the details of this POST request. The URL is '[[baseUri]] /api/upload-excel'. The request body is set to 'form-data' and contains three fields: 'file' (value: 'userstories.xlsx'), 'dataType' (value: 'userstories'), and 'sheetName' (value: 'stories'). The response status is '200 OK' with a response time of 345 ms and a body size of 645 B. The response body is a JSON object indicating success and providing details about the conversion process.

Request Details:

- Method: POST
- URL: [[baseUri]] /api/upload-excel
- Body Type: form-data
- Fields:
 - file: userstories.xlsx (Excel file containing user stories)
 - dataType: userstories (Type of data: 'testcases' or 'userstories')
 - sheetName: stories (Name of the Excel sheet to convert (optional))

Response Details:

- Status: 200 OK
- Time: 345 ms
- Size: 645 B
- Body (JSON):

```
{  "success": true,  "message": "User stories file converted successfully",  "outputFile": "stories-1782378935383.json",  "output": "Converting 196 rows from sheet '\u0026quot;stories\u0026quot;...\u0026quot; Converted 196 user stories from '\u0026quot;stories\u0026quot; into C:/Users/Manddhini Shreekanth/Downloads/rag-mongo-demo-v9/rag-mongo-demo-v9/src/data/stories-1782378935383.json\u0026quot;. Filtered out: 0 empty rows\u0026quot;"}
```

Getting the List of Available files

The screenshot shows the Postman interface for a REST client. The left sidebar displays the same collection as the previous screenshot, but now 'GET 3. List Available Files' is selected. The main panel shows the details of this GET request. The URL is '[[baseUri]] /api/files'. The response status is '200 OK' with a response time of 5 ms and a body size of 741 B. The response body is a JSON array containing two file objects with their names, paths, sizes, modification times, and types.

Request Details:

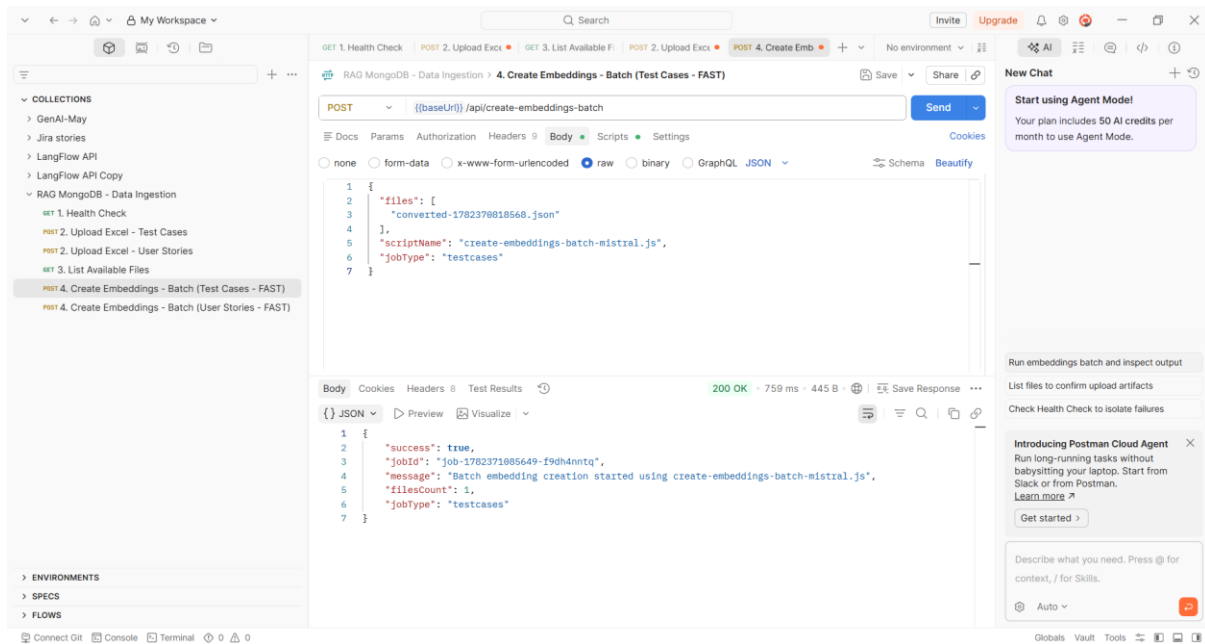
- Method: GET
- URL: [[baseUri]] /api/files

Response Details:

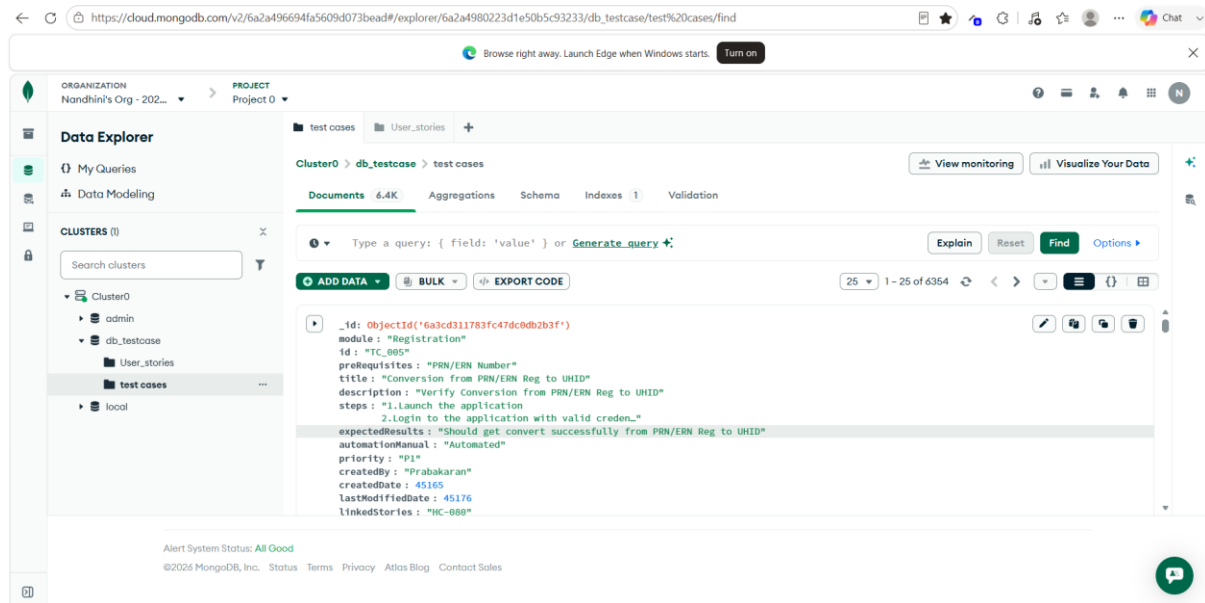
- Status: 200 OK
- Time: 5 ms
- Size: 741 B
- Body (JSON):

```
[  {    "name": "converted-1782378618568.json",    "path": "C:\\Users\\Manddhini Shreekanth\\Downloads\\rag-mongo-demo-v9\\rag-mongo-demo-v9\\src\\data\\converted-1782378618568.json",    "size": 6698526,    "modified": "2026-06-25T07:00:20.001Z",    "type": "json"  },  {    "name": "stories-1782378935383.json",    "path": "C:\\Users\\Manddhini Shreekanth\\Downloads\\rag-mongo-demo-v9\\rag-mongo-demo-v9\\src\\data\\stories-1782378935383.json"  }]
```

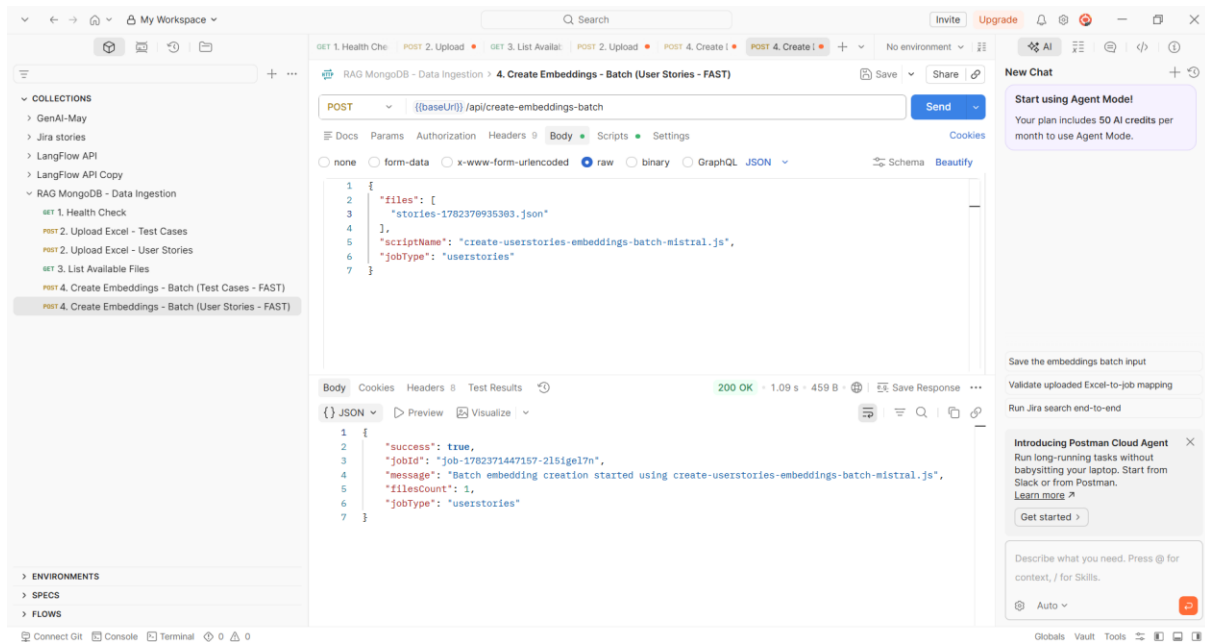
Create Embedding test cases using API postman



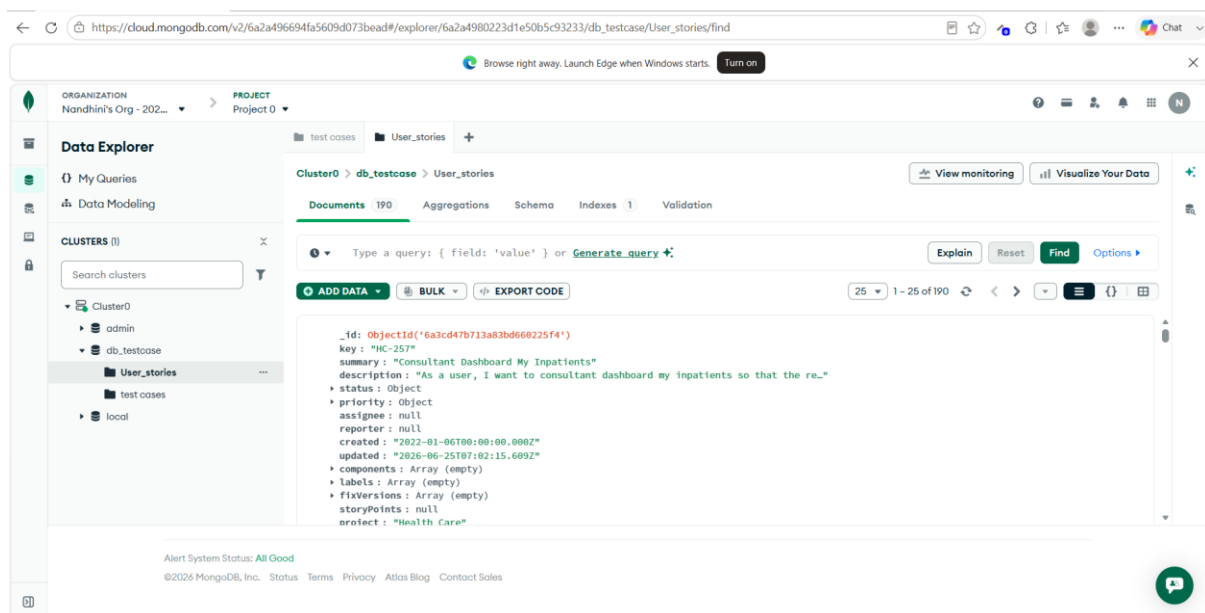
Verified the test cases are displayed in Mango DB



Create Embedding User Stories using API postman



Verified the User Stories are displayed in Mango DB



2. Validating the Retrieval through API

Ans: - Verified the Vector Search

The screenshot shows a Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "RAG MongoDB - Complete Ingestion & Retrieval" and "RETRIEVAL". The "RETRIEVAL" collection is expanded, showing "POST 09. Vector Search" selected. The main panel displays the details of this request, which is a POST to "({baseUri})/api/search". The request body is a JSON object with the following structure:

```
{  "query": "[[Share Diagnostic Reports with Patients via WhatsApp]]",  "limit": 10,  "filters": {    "module": "",    "priority": "",    "risk": "",    "automationManual": ""  },  "steps": [    {      "id": "6a3cd316783fc47dc8dh2c83",      "module": "AHC",      "id": "TC_2736",      "title": "AHCSummary ->AHCSummary ->Diagnostic & Imaging report",      "description": "Verify Diagnostic & Imaging report",      "steps": "1. Launch the application\r\n2. Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select Doctors and wards from the drop down.\r\n4. Click on Patient Record Link\r\n5. Click on Patient Identifier Number\r\n6. Click on Report Link Available against the patient",      "expectedResults": "The Report should be displayed",      "automationManual": "Manual",      "priority": "P3",      "createdBy": "Sumitha",      "createdAt": "45185",      "lastModifiedDate": "45200",      "risk": "Low",      "version": "v19.0",      "type": "UI",      "createdAt": "2826-06-25T07:06:57.776Z",      "score": "0.9074357748831616"    }  ],  "expectedResults": "The Report should be displayed",  "automationManual": "Manual",  "priority": "P3",  "createdBy": "Sumitha",  "createdAt": "45185",  "lastModifiedDate": "45200",  "risk": "Low",  "version": "v19.0",  "type": "UI",  "createdAt": "2826-06-25T07:06:57.776Z",  "score": "0.9074357748831616"}
```

The response is a 200 OK status with a response time of 2.80s and a size of 9.31 KB. The response body is a JSON object with the following structure:

```
{  "id": "6a3cd316783fc47dc8dh2c83",  "module": "AHC",  "id": "TC_2736",  "title": "AHCSummary ->AHCSummary ->Diagnostic & Imaging report",  "description": "Verify Diagnostic & Imaging report",  "steps": "1. Launch the application\r\n2. Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select AHC from drop down.\r\n4. Navigate to Functions -> AHCSummary ->AHCSummary.\r\n5. Select date or Enter UNID/AHC No and click on 'Go' button.\r\n6. Click on AHC Number.\r\n7. Click on Diagnostic & Imaging link\r\n8. Observe the results\r\n9. Click on download or print icon",  "expectedResults": "Diagnostic & Imaging details should be displayed",  "automationManual": "Automated",  "priority": "P3",  "risk": "Low",}
```

Verified BM25 Full – Text Search

The screenshot shows a Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "RAG MongoDB - Complete Ingestion & Retrieval" and "RETRIEVAL". The "RETRIEVAL" collection is expanded, showing "POST 10. BM25 Full-Text Search" selected. The main panel displays the details of this request, which is a POST to "({baseUri})/api/search/bm25". The request body is a JSON object with the following structure:

```
{  "query": "[[Share Diagnostic Reports with Patients]]",  "limit": 10,  "filters": {    "module": "",    "priority": "",    "risk": ""  },  "fields": [    "id",    "title",    "description",    "steps"  ],  "expectedResults": "The Report should be displayed",  "automationManual": "Manual",  "priority": "P3",  "createdBy": "Sumitha",  "createdAt": "45185",  "lastModifiedDate": "45200",  "risk": "Low",  "version": "v19.0",  "type": "UI",  "createdAt": "2826-06-25T07:06:57.776Z",  "score": "0.9074357748831616"}
```

The response is a 200 OK status with a response time of 2.16s and a size of 8.27 KB. The response body is a JSON object with the following structure:

```
{  "id": "6a3cd316783fc47dc8dh2c83",  "module": "AHC",  "id": "TC_2736",  "title": "AHCSummary ->AHCSummary ->Diagnostic & Imaging report",  "description": "Verify Diagnostic & Imaging report",  "steps": "1. Launch the application\r\n2. Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select AHC from drop down.\r\n4. Navigate to Functions -> AHCSummary ->AHCSummary.\r\n5. Select date or Enter UNID/AHC No and click on 'Go' button.\r\n6. Click on AHC Number.\r\n7. Click on Diagnostic & Imaging link\r\n8. Observe the results\r\n9. Click on download or print icon",  "expectedResults": "Diagnostic & Imaging details should be displayed",  "automationManual": "Automated",  "priority": "P3",  "risk": "Low",}
```

Verified Hybrid Search (BM25+Vector)

POST 09. Vector Search • POST 10. BM25 Full-Text Search • **POST 11. Hybrid Search (BM25 + Vector)**

Send

Body

```
1 {
2   "query": "[[share total admission per period]]",
3   "limit": "[[searchLimit]]",
4   "bm25Weight": 0.4,
5   "vectorWeight": 0.6,
6   "filters": {
7     "module": "",
8     "priority": "",
9     "risk": ""
10  }
11 }
```

Body

```
15 {
16   "_id": "6a3cd311783fc478c8db2b4d",
17   "module": "AT(Admission Module)",
18   "id": "TC_1896",
19   "title": "Total admissions within a period",
20   "description": "To verify user able to segregate Total admissions within a period",
21   "steps": "1. Launch the application\r\n2.Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select AT from the drop down\r\n4.Click on Function-> Others-> Reports-> Total admissions within a period\r\n5.Select From Date, To Date and Click on Report Button",
22   "expectedResults": "User should be able to view IP details based on Total admissions within a period",
23   "automationManual": "Automated",
24   "priority": "P1",
25   "risk": "Critical",
26   "createdAt": "2026-06-25T07:04:49.789Z",
27   "bm25Score": 32.04499280151367,
28   "bm25ScoreNormalized": 1.
29 }
```

200 OK · 3.86 s · 11.44 KB

JSON

Preview

Visualize

Run Hybrid Search end-to-end

Align Hybrid weights with BM25/Vector

Add assertions for Hybrid scoring

Introducing Postman Cloud Agent

Run long-running tasks without babysitting your laptop. Start from Slack or from Postman.

Learn more

Get started

Describe what you need. Press @ for context, / for Skills.

Auto

Globals Vault Tools

Verified rerank search (Groq AI LLM Reranking)

POST 09. Vector Search • POST 10. BM25 Full-Text Search • POST 11. Hybrid Search (BM25 + Vector) • **POST 12. Rerank Search (Groq AI LLM Reranking)**

Send

Body

```
1 {
2   "query": "[[user able to generate SpecialNotes Report]]",
3   "limit": 10,
4   "rerankTopK": 50,
5   "filters": {
6     "module": "",
7     "priority": "",
8     "risk": ""
9   }
10 }
```

Body

```
18 {
19   "results": [
20     {
21       "_id": "6a3cd311783fc478c8db2b4d",
22       "module": "AT(Admission Module)",
23       "id": "TC_1825",
24       "title": "SpecialNotes",
25       "description": "To verify the user able to generate SpecialNotes Report",
26       "steps": "1.Launch the application\r\n2.Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select AT from the drop down.\r\n4.Navigate Functions -> Others -> \r\n5.Select SpecialNotes link.\r\n6.Enter From date and To date. \r\n7.Select Location.\r\n8.Click on 'REPORT' button.",
27       "expectedResults": "SpecialNotes PDF Report should generate.",
28       "priority": "P1",
29       "risk": "Critical",
30       "bm25Score": 26.14634895324787
31     }
32   ]
33 }
```

200 OK · 7.55 s · 7.29 KB

JSON

Preview

Visualize

Execute rerank search with baseline

Run hybrid+assert consistency checks

Capture failing edge cases

Introducing Postman Cloud Agent

Run long-running tasks without babysitting your laptop. Start from Slack or from Postman.

Learn more

Get started

Describe what you need. Press @ for context, / for Skills.

Auto

Globals Vault Tools

Verified Preprocess Query

The screenshot displays the Postman interface for a REST client. The left sidebar shows a collection of requests under 'RAG MongoDB - Complete Ingestion & Retrieval'. The main panel shows a POST request to 'POST 13. Preprocess Query' with a body containing a JSON object. The response is a 200 OK status with a JSON body.

Request:

```
POST {{baseUri}}/api/search/preprocess
```

Body (raw):

```
{
  "query": "{{Share nutrition plan}}",
  "options": {
    "enableAbbreviations": true,
    "enableSynonyms": true,
    "maxSynonymVariations": 5,
    "smartExpansion": false,
    "preserveTestCaseIds": true
  }
}
```

Response (JSON):

```
{
  "abbreviationExpanded": "share nutrition plan",
  "synonymExpanded": [
    "share nutrition plan"
  ],
  "metadata": {
    "testCaseIds": [],
    "abbreviationMappings": [],
    "synonymMappings": [],
    "tokens": [
      "share",
      "nutrition",
      "plan"
    ]
  },
  "processingTime": 3,
  "steps": {
    "normalization": true,
    "abbreviationExpansion": true,
    "synonymExpansion": true
  }
}
```

Verified Analyze Query

The screenshot displays the Postman interface for a REST client. The left sidebar shows a collection of requests under 'RAG MongoDB - Complete Ingestion & Retrieval'. The main panel shows a POST request to 'POST 14. Analyze Query' with a body containing a JSON object. The response is a 200 OK status with a JSON body.

Request:

```
POST {{baseUri}}/api/search/analyze
```

Body (raw):

```
{
  "query": "{{Share consultation from booked status}}",
  "analysis": {
    "hasTestCaseIds": false,
    "testCaseIds": [],
    "hasAbbreviations": false,
    "abbreviations": [],
    "hasSynonymOpportunities": true,
    "synonymOpportunities": [
      {
        "term": "consultation",
        "synonyms": [
          "appointment",
          "visit",
          "examination"
        ]
      }
    ]
  },
  "position": 1
}
```

Response (JSON):

```
{
  "original": "{{Share consultation from booked status}}",
  "normalized": "share consultation from booked status",
  "analysis": {
    "hasTestCaseIds": false,
    "testCaseIds": [],
    "hasAbbreviations": false,
    "abbreviations": [],
    "hasSynonymOpportunities": true,
    "synonymOpportunities": [
      {
        "term": "consultation",
        "synonyms": [
          "appointment",
          "visit",
          "examination"
        ]
      }
    ]
  },
  "position": 1
}
```


Verified Summarize results(Consize)

The screenshot shows the Postman REST client interface. The left sidebar displays a collection of API collections, with 'POST-RETRIEVAL' selected. The main panel shows a POST request to 'RAG MongoDB - Complete Ingestion & Retrieval' with the endpoint '/api/search/summarize'. The request body is a JSON object with the following structure:

```
{  "id": "TC_0001",  "module": "Login",  "title": "Verify valid user login",  "description": "Test that a valid user can log in successfully",  "steps": "1. Navigate to login page\n2. Enter valid credentials\n3. Click Login",  "expectedResults": "User is redirected to dashboard",  "priority": "High",  "risk": "High",  "automationManual": "Automation"}
```

The response is a 200 OK status with a JSON body:

```
{  "success": true,  "summary": "The search results for 'login functionality test cases' provide a starting point for testing login functionality. The results include a test case, TC_0001, titled 'Verify valid user login' which is part of the Login module. This test case aims to verify that a valid user can log in successfully. This test case is a fundamental aspect of ensuring that the login functionality works as expected. It covers the basic requirement of allowing authorized users to access the system. By including this test case, developers can ensure that the login process is functioning correctly and that valid users are not denied access. To further enhance the testing of login functionality, additional test cases can be created to cover various scenarios, such as invalid username or password, account lockout, and password recovery. These test cases can help identify potential issues and ensure that the login functionality is robust and secure. Overall, the search results provide a foundation for testing login functionality, and by expanding on this initial test case, developers can create a comprehensive set of test cases to ensure the reliability and security of the login process.",  "summaryType": "concise",  "resultCount": 1,  "model": "llama-3.3-70b-versatile"}
```

Verified Summarize Results (Detailed)

The screenshot shows the Postman REST client interface. The left sidebar displays a collection of API collections, with 'POST-RETRIEVAL' selected. The main panel shows a POST request to 'RAG MongoDB - Complete Ingestion & Retrieval' with the endpoint '/api/search/summarize'. The request body is a JSON object with the following structure:

```
{  "id": "TC_0001",  "module": "Login",  "title": "Verify valid user login",  "description": "Test that a valid user can log in successfully",  "steps": "1. Navigate to login page\n2. Enter valid credentials\n3. Click Login",  "expectedResults": "User is redirected to dashboard",  "priority": "High",  "risk": "High",  "automationManual": "Automation"}
```

The response is a 200 OK status with a JSON body:

```
{  "success": true,  "summary": "**Detailed Analysis of Search Results: 'Login Functionality Test Cases'**\n\nThe search query 'login functionality test cases' yielded a single relevant result, which is a test case for verifying valid user login. Here's a breakdown of the analysis:\n\n**Statistics:**\n\n- Number of results: 1\n- Relevance: 100% (the single result is directly related to the search query)\n\n**Module:** Login (indicating that the result is specific to the login functionality)\n\n**Common Themes:**\n\n- User authentication: The result focuses on testing the login functionality for a valid user, which is a critical aspect of user authentication.\n- Test case design: The result provides a specific test case (TC_0001) with a clear title, module, and description, demonstrating a structured approach to testing login functionality.\n\n**Relevance Insights:**\n\n- Direct relevance: The single result is directly related to the search query, indicating that the search term is specific and targeted.\n- Limited scope: The result only covers a single test case, suggesting that the search query may need to be broadened or modified to retrieve more comprehensive results.\n- Importance of login testing: The presence of a specific test case for valid user login highlights the importance of testing login functionality to ensure secure and reliable user authentication.\n\n**Actionable Recommendations:**\n\n- Broaden the search query: To retrieve more results, consider modifying the search query to include related terms, such as
```


Verified Deduplicate Results

The screenshot shows the Postman interface for the 'deduplicate' endpoint. The request is a POST to '[[baseUrl]] /api/search/deduplicate'. The response is a 200 OK status with a JSON body. The JSON body contains a 'deduplicated' array with two objects, each with 'id' and 'title' fields. The first object has 'id': 'TC_0001' and 'title': 'Verify valid user login'. The second object has 'id': 'TC_0002' and 'title': 'Verify valid user login flow'. The response also includes a 'duplicates' array (empty) and a 'stats' object with 'originalCount': 2, 'deduplicatedCount': 2, 'duplicatesRemoved': 0, and 'reductionPercentage': '0.0'.

```
POST [[baseUrl]] /api/search/deduplicate
```

```
{
  "deduplicated": [
    {
      "id": "TC_0001",
      "title": "Verify valid user login"
    },
    {
      "id": "TC_0002",
      "title": "Verify valid user login flow"
    }
  ],
  "duplicates": [],
  "stats": {
    "originalCount": 2,
    "deduplicatedCount": 2,
    "duplicatesRemoved": 0,
    "reductionPercentage": "0.0"
  }
}
```

Verified by testing the AI Prompt (GROQ)

The screenshot shows the Postman interface for the 'test-prompt' endpoint. The request is a POST to '[[baseUrl]] /api/test-prompt'. The response is a 200 OK status with a JSON body. The JSON body contains a 'success' field set to true, a 'response' field with a long string of text, and a 'tokens' object with 'prompt': 115, 'completion': 380, and 'total': 495.

```
POST [[baseUrl]] /api/test-prompt
```

```
{
  "success": true,
  "response": "Edge-case test cases\n\n1 | Test case title | Why it's an edge case\n\n---\n\n1 | Verify login with empty password | Tests the boundary condition of a missing (zero-length) credential - an input that is at the extreme end of the valid domain.\n\n2 | Verify login with SQL injection | Uses a malicious, atypical input to probe how the system handles unexpected, potentially dangerous data. This is a classic security-edge scenario.\n\n3 | Verify valid user login - normal, expected behavior.\n\n4 | Verify dashboard loads after login - normal post-login flow.\n\nSo, the edge-case coverage is provided by test cases 1, 2, and 3.",
  "tokens": {
    "prompt": 115,
    "completion": 380,
    "total": 495
  }
}
```

Verified Get Metadata (Distinct Filter Values)

The screenshot shows the Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "UTILITIES" which contains "19. Get Metadata (Distinct Filter Values)". The main panel shows a GET request to `{{baseUrl}}/api/metadata/distinct` with a status of "200 OK" and a response time of "1.89 s". The response body is a JSON object:

```
{
  "success": true,
  "metadata": {
    "modules": [
      "AHC",
      "AT(Admission Module)",
      "Admin",
      "BB(Blood Bank)",
      "Digital GP",
      "Digital dietician",
      "Doctors-Wards",
      "Dynamic Forms",
      "ES(Appointments)",
      "Emergency",
      "Human Resources (HR)",
      "IP Billing",
      "IP Digital",
      "IP digital"
    ]
  }
}
```

Verified by Getting Latest Test Case ID

The screenshot shows the Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "UTILITIES" which contains "20. Get Latest Test Case ID". The main panel shows a GET request to `{{baseUrl}}/api/testcases/latest-id` with a status of "200 OK" and a response time of "1.66 s". The response body is a JSON object:

```
{
  "success": true,
  "latestId": 6354,
  "nextId": 6355,
  "nextTestCaseId": "TC_6355",
  "totalTestCases": 6354
}
```

The screenshot displays the Postman REST client interface. On the left, a sidebar shows the API collection structure with folders like 'Jira stories', 'LangFlow API', and 'RAG MongoDB - Complete Ingestion & Retrieval'. The main area shows a GET request to 'http://localhost:3000/api/env'. The response is a 200 OK status with a JSON body containing various configuration parameters. The interface also includes a 'New Chat' button and a 'Postman Cloud Agent' sidebar.

```

{
  "MONGODB_URI": "mongodb+srv://Nandhini:bgyPun8fyrw590Kig@cluster0.zrktr1a.mongodb.net/?appName=cluster2",
  "DB_NAME": "db_testcase",
  "COLLECTION_NAME": "test cases",
  "VECTOR_INDEX_NAME": "vector_index",
  "BM25_INDEX_NAME": "default",
  "USER_STORIES_COLLECTION_NAME": "user_stories",
  "USER_STORIES_VECTOR_INDEX_NAME": "vector_index_1",
  "USER_STORIES_BM25_INDEX_NAME": "bm25_user_stories",
  "MISTRAL_API_KEY": "h8u8uxqk9McyEOcNEr8Z6ShoA890id",
  "MISTRAL_EMBEDDING_MODEL": "mistral-embed",
  "GROQ_API_KEY": "gsk_CtpISofIT0J9tC4d4483W6dyb3FY5sWFTv0ob3GvIHU6V9K8aa",
  "GROQ_RERANK_MODEL": "openai/gpt-oss-120b",
  "GROQ_SUMMARIZATION_MODEL": "llama-3.3-70b-versatile"
}

```